



PI DELTA CHAPTER

2025

# RushBot Project Report



PROJECT MANAGER:  
DIEGO SOLORZANO

UNIVERSITY OF CALIFORNIA,  
IRVINE

## Executive Summary

RushBot is an autonomous, interactive robot designed to improve Rush turnout by creating a dynamic and engaging experience at tabling and booth events. Traditional recruitment methods often rely on passive engagement, but RushBot introduces an innovative approach to draw in potential new members. By combining robotics, artificial intelligence, and automation, this project aims to increase event participation and improve the fraternity's outreach efforts while also serving as a way to develop our members' skills through a hands-on engineering experience.

At its core, RushBot is a rover-like robot equipped with 6 wheels powered by its own DC motor and a suspension system making it capable of driving through various terrains. By utilizing LiDAR and Simultaneous Localization and Mapping (SLAM) technology, it is able to autonomously navigate event spaces. It features a 2-degrees-of-freedom (2DOF) manipulator capable of grabbing and distributing rush flyers to interested individuals, making the recruitment process more interactive. Additionally, RushBot is integrated with an AI-powered chatbot that can answer questions about rush, fraternity values, and upcoming events, providing real-time engagement with potential recruits. The robot's electrical system features both high and low-voltage systems with built-in safety circuitry to support all of the robot's needs.

This project is intended to be continuously improved and expanded upon in the coming years, adapting to new technologies and recruitment strategies. By establishing RushBot as a long-term tool for engagement, we hope to set a new standard for how our fraternity interacts with potential members and presents itself at public events.

## Table of Contents

|                                                   |    |
|---------------------------------------------------|----|
| Executive Summary .....                           | 1  |
| Budget Breakdown .....                            | 2  |
| Chapter 1: Budget Breakdown .....                 | 3  |
| Chapter 2: Electrical Subteam .....               | 5  |
| Chapter 3: Mechanical Subteam .....               | 11 |
| Chapter 4: Controls Subteam .....                 | 31 |
| Chapter 5: Impact Assessment and Conclusion ..... | 38 |

## Chapter 1: Budget Breakdown

| Electrical Budget Breakdown          |         |          |                 |
|--------------------------------------|---------|----------|-----------------|
| Item                                 | Price   | Quantity | Total           |
| 5V Power converter for Pi            | \$8.99  | 1        | \$8.99          |
| nema17 stepper motor                 | \$13.99 | 2        | \$27.98         |
| magnetic encoder - 3 pack            | \$9.99  | 1        | \$9.99          |
| Arduino power Converter              | \$8.99  | 1        | \$8.99          |
| 12V Buck Converter                   | \$8.99  | 1        | \$8.99          |
| Battery voltage sensor               | \$9.99  | 1        | \$9.99          |
| Battery current sensor               | \$11.58 | 1        | \$11.58         |
| Wheel servos                         | \$43.00 | 1        | \$43.00         |
| Power switch -30A                    | \$7.99  | 1        | \$7.99          |
| mux - 6 pack                         | \$9.99  | 1        | \$9.99          |
| 5V Power Convertor for Servos        | \$18.69 | 1        | \$18.69         |
| USB hub                              | \$14.00 | 1        | \$14.00         |
| 12V power supply                     | \$11.99 | 1        | \$11.99         |
| Arduino mega                         | \$22.99 | 1        | \$22.99         |
| Limit switches                       | \$4.99  | 1        | \$4.99          |
| EVE 26V 18650 2550mAh 7.5A Battery   | \$1.70  | 32       | \$54.40         |
| BMS                                  | \$8.07  | 1        | \$8.07          |
| 12V DC Motor with Wheel Encoder      | \$5.86  | 6        | \$35.16         |
| 12 DC Motor Drivers -4pc             | 11.99   | 1        | 11.99           |
| INRFZ44N Mosfet                      | \$1.36  | 1        | \$1.36          |
| Stepper Motor Driver TB6600 4A 9-42V | \$9.98  | 2        | \$19.96         |
| Lidar Sensor                         | \$99.99 | 1        | \$99.99         |
| <b>Total</b>                         |         |          | <b>\$451.08</b> |



| Mechanical Budget Breakdown                           |         |          |                 |
|-------------------------------------------------------|---------|----------|-----------------|
| Item                                                  | Price   | Quantity | Total           |
| M2 Screw Kit                                          | \$9.90  | 1        | \$9.90          |
| M2 (3.2 OD x 4 L) Heat Insert Kit                     | \$8.99  | 1        | \$8.99          |
| 4pcs 300mm T Slot                                     | \$17.00 | 1        | \$17.00         |
| 4pcs 500mm T Slot 2020                                | \$30.00 | 1        | \$30.00         |
| Corner Brackets, Screws, T-Slot Fasteners             | \$22.00 | 1        | \$22.00         |
| Screw Kit (M3~M6)                                     | \$13.98 | 1        | \$13.98         |
| 8x22x7mm bearings - 10 pack                           | \$8.00  | 1        | \$8.00          |
| 8x22x7mm flanged bearings - 10 pack                   | \$7.97  | 1        | \$7.97          |
| M8 Metric Screw Set                                   | \$14.00 | 1        | \$14.00         |
| M8-1.25 x 140mm Length Fully Threaded Rod             | \$5.99  | 1        | \$5.99          |
| SP Large Vacuum Silicone Suction Cup                  | \$3.05  | 1        | \$3.05          |
| 12mm PVC Hose Pipe Transparent Pipe                   | \$3.44  | 1        | \$3.44          |
| PTEG Filament                                         | \$19.99 | 3        | \$59.97         |
| Spring                                                | \$8.69  | 1        | \$8.69          |
| M6 x 35mm Flanged Hex Head Bolts Screws               | \$8.69  | 1        | \$8.69          |
| 606-2RS Deep Groove Ball Bearing Double Sealed 180016 | \$8.29  | 1        | \$8.29          |
| Carbon Fiber Tubes Matte Surface                      | \$32.98 | 1        | \$32.98         |
| 4Pcs 4mm Flange Coupling Connector                    | \$7.99  | 1        | \$7.99          |
| 6008 Deep Groove Ball Bearings 40mm                   | \$7.99  | 4        | \$31.96         |
| LMK20UU Square Flange Linear Ball Bearings            | \$16.79 | 1        | \$16.79         |
| <b>Total</b>                                          |         |          | <b>\$319.68</b> |

|                    |                 |
|--------------------|-----------------|
| <b>Grand Total</b> | <b>\$770.76</b> |
|--------------------|-----------------|

## Funding

The Rushbot project has been awarded a total of \$750 in funding by the engineering student council of UCI. The remaining fund is expected to be covered by the fraternity.

## Chapter 2: Electrical Subteam

Name: Luke Vargas

Email: [lukevargas@gmail.com](mailto:lukevargas@gmail.com)

- Creating Wiring Schematics and designed several components on Altium Designer for Arduino and peripherals and the power circuit
- Integrated electrical components

---

Name: Dalena Nguyen

Email: [dalenanguyenwork@gmail.com](mailto:dalenanguyenwork@gmail.com)

- Helped create schematic for ARDUINO and PI wiring diagram

---

Name: Arim Song

Email: [arimsong0430@gmail.com](mailto:arimsong0430@gmail.com)

- Helped combine Arduino and Rasp pi wiring diagrams by linking key power components for full electrical schematic

---

Name: Keiko Yamamuro

Email: [keiko.yamamuro@gmail.com](mailto:keiko.yamamuro@gmail.com)

- Performed power calculations and helped to build a battery using 18650 cells

---

Name: David Culciar

Email: [davidculciar@gmail.com](mailto:davidculciar@gmail.com)

- Assisted in battery design and testing

---

Name: Shana Chao

Email: [shanaxchao@gmail.com](mailto:shanaxchao@gmail.com)

- Integrated and tested electrical components, diagnosing and resolving performance issues
- 

### *Chapter 2.1: Voltage System Hierarchy*

RushBot is powered by a 14.8V 20400mAh battery comprised of 32 18650 Lithium Ion battery cells arranged in a 4s8p configuration. Since the nominal voltage of each cell is 3.7V, connecting 4 in series would give us an overall nominal voltage of 14.8V, with 16V when the battery is fully charged. Connecting 8 in parallel allows us to

increase our battery life and reach 20400mAh. Lithium-ion was chosen as opposed to Lithium-polymer due to its more level discharger over time. This allows for more time at nominal voltage and this results in increased efficiency. A traditional 12V battery was found to not provide enough power to the system due to a voltage drop after the voltage regulator. Constructing a battery with a higher initial voltage proves to be more efficient as it requires less power to step down compared to stepping up the voltage to our required 12V. The final power calculations of the rover's battery life are shown below in Table 1. Current draw for each state is gathered from the datasheets of all components and the calculations are found using the following equation:

$$\text{Battery Life} = \frac{\text{Total Amp Hours}}{\text{Current Draw}}$$

|                  | Idle  | Active | Stall |
|------------------|-------|--------|-------|
| Run Time (Hours) | 33.12 | 4.13   | 0.79  |

Table 1: Battery Life for three different states

RushBot has a power management system in order to ensure safety and stable operation, which utilizes a switch, one 12 V step-down buck converter, and two 12V-to-5V regulators. The battery itself features a 30A battery management system ensuring even discharge and charging of each cell.

The battery switch is the first component that the battery is connected to and serves as a reliable method to disconnect power from the entire system. This controlled power activation ensures both safety and efficiency as it prevents potential component damage and overheating in the event of malfunction.

Rushbot uses two types of regulators: a 12 V step-down regulator and two 12V-to-5V regulators. Regulators help protect the system by ensuring that each component receives the correct voltage, preventing overvoltage that could damage the electronics. The 12V regulator provides a stable voltage for the NEMA 17 stepper motors. Meanwhile, the 12V to 5V regulators step down the voltage to 5V, delivering 3A to the Raspberry Pi and 8A to the servo motors, ensuring both components operate safely at their required voltage levels. The wiring diagram for the voltage systems is shown below in Figure 1.

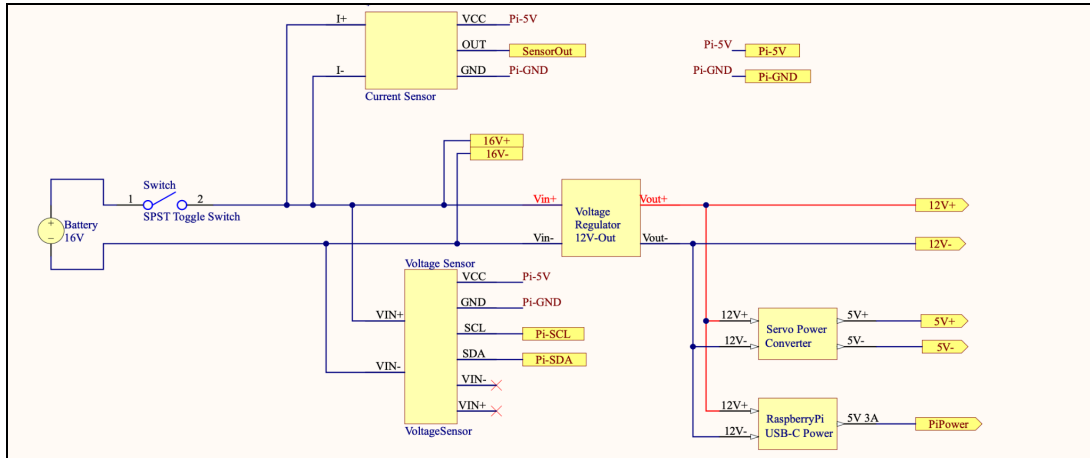


Figure 1: Power Wiring Diagram

## Chapter 2.2: Motors and Components

### Motor Control

RushBot uses a total of 6 12V brushless DC motors, 2 NEMA 17 stepper motors, 4 High Torque servo motors, and 1 vacuum pump motor for a total of 13 motors controlled by the Arduino Mega. A Mega is chosen due to its extra PWM signal pins which all of the motors operate on.

3 L298N drivers control all 12V DC Motors. Each unit acts as dual drivers allowing for 2 motors to be controlled by each one. Due to the varying turning radius of each motor, the DC motors must be controlled in 4 different to account for the varying speed of each wheel. For this reason, 1 of the 3 drivers will individually control 2 motors separately while the other drivers control their motors together. In total, this allows for 4 different speeds. Additionally, each DC motor is equipped with onboard encoders that are read via digital pins.

2 TB6600 drivers control the stepper motors. These drivers, while bulky, are chosen due to their excellent heat dissipation, which is a common issue seen in most stepper motor drivers. This was especially crucial for our design as these stepper motors would be under high load from moving the arm, resulting in a higher average current draw. Attached behind the stepper motors are AS5600 magnetic encoders used to accurately track the motors' rotation. This data is communicated through an I2C protocol. However since both drivers have the same address, a multiplexer is needed in order to properly differentiate the two when communicating to the Arduino.

The vacuum Pump Motor is controlled using a logic-level NMOS switch circuit. The INRFZ44N NMOS channel mosfet was chosen as the switch due to its triode operation region when a voltage of 5V is applied at the gate. This characteristic of the mosfet makes it ideal when interfacing with Arduino since its digital pins operate at the 5V logic level. A 10 ohm gate resistor is used to limit the voltage at the gate and a 100K ohm pull-down resistor is used to provide a return path for the current in order to properly switch off the mosfet.

Lastly, the 4 servos are directly controlled by the Arduino's PWM pins, and 2 switches between the joints of the arm to prevent over-extension and protect circuitry and structure. All Arduino connections can be seen in Figures 2-4.

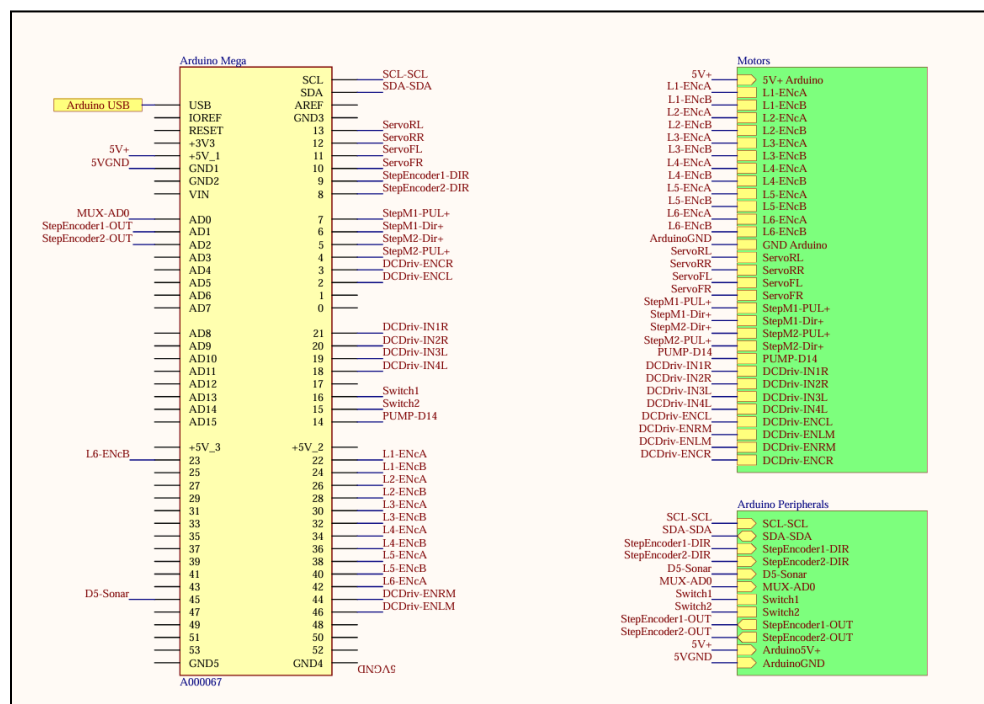


Figure 2: Arduino input/output connections

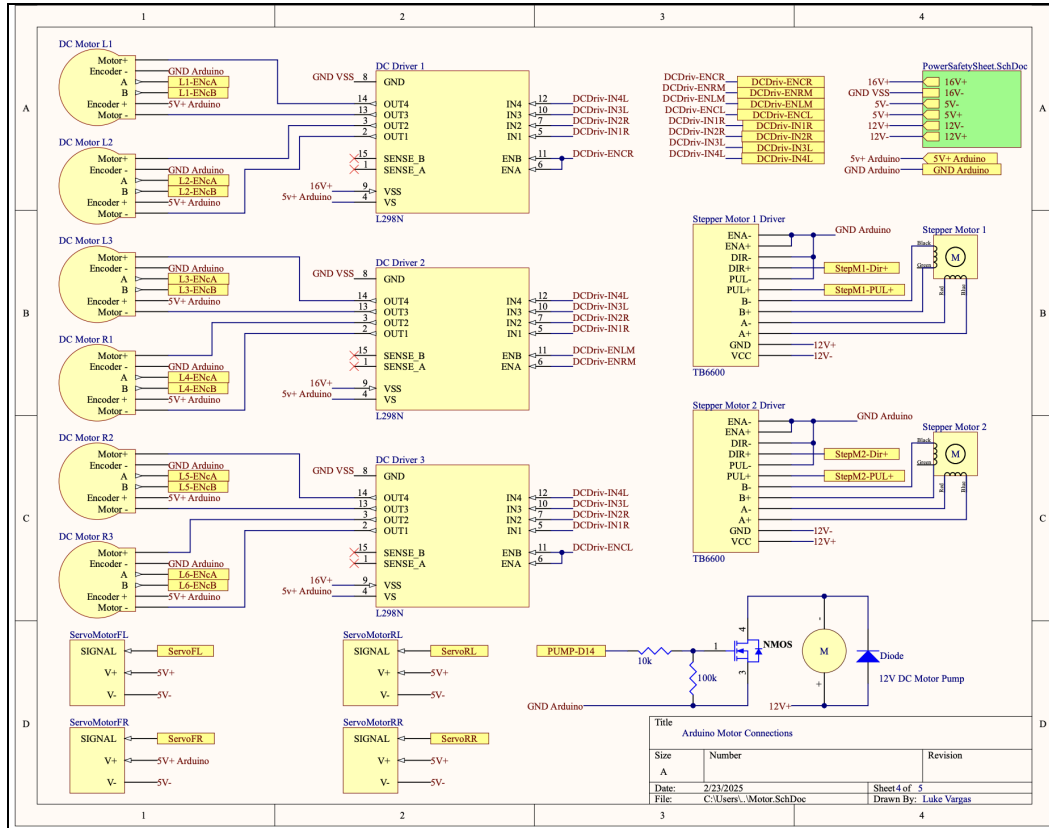


Figure 3: Motor Connection Wiring Diagram

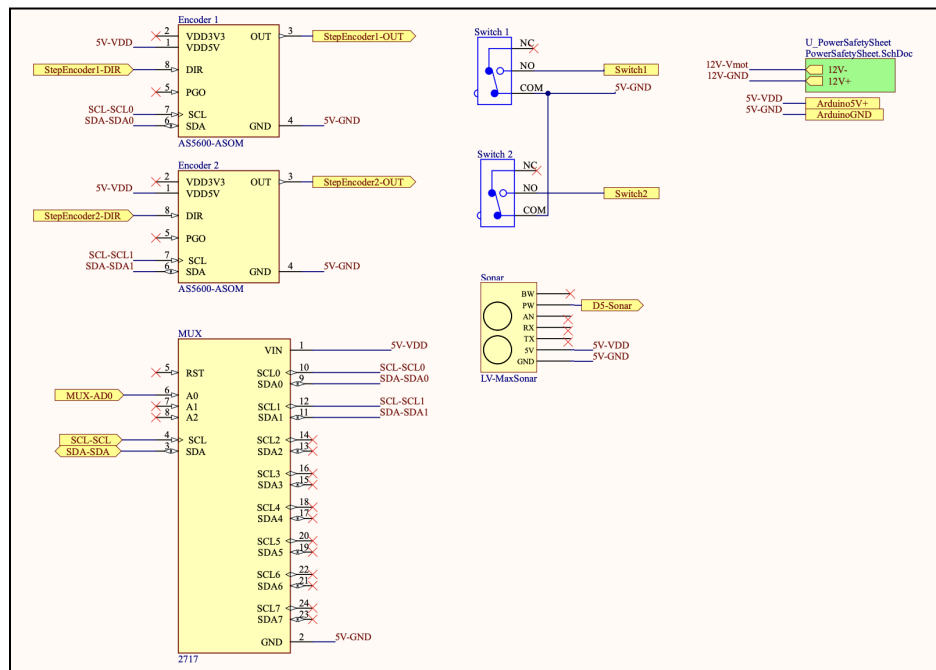


Figure 4: Encoder, Mux, and Limit Switch Wiring Diagram

## Sensors

The Low-voltage system is comprised of various additional sensors throughout the robot for proper and safe operation.

Connected to the Raspberry Pi are dedicated voltage and current sensors meant to probe the battery in order to digitally monitor the power consumption of the rover at any given time. This is important as it acts as an extra layer of safety to prevent access current draw above the rated limit. The voltage sensor also allows us to monitor the battery life of the battery ensuring the voltage doesn't fall below a safe operating limit. Once the voltage falls below this limit, this lets us know the battery must be recharged. Additionally, an inertial measurement unit (IMU), GPS, and Lidar sensor are connected to the Pi as well and are meant to be interfaced with the ROS control system for rover control. The wiring for these sensors is shown below in Figure 5.

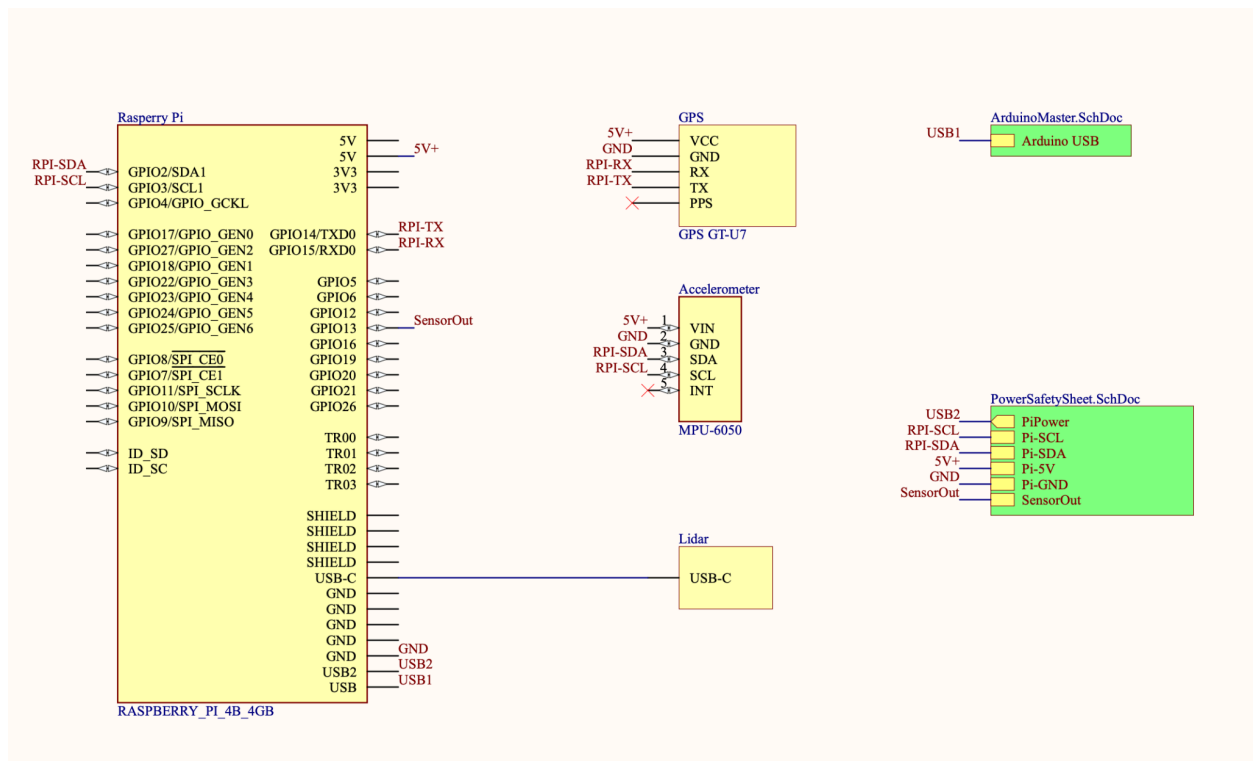


Figure 5: Raspberry Pi Sensors Wiring Diagram

## Chapter 3: Mechanical Subteam

Name: Tyler Buffa

Email: [tylerbuffa@gmail.com](mailto:tylerbuffa@gmail.com)

- Designed and fabricated the flyer dispenser
- Developed MATLAB script to optimize dispenser linkage lengths for maximum dispensing height and minimum stowed height
- Conducted simulated flyer dispenser tests using Simulink to find torque requirements and gear ratio

---

Name: Dayeon Yoon

Email: [diane040611@gmail.com](mailto:diane040611@gmail.com)

- Designed and manufactured rocker-bogie suspension for the rover
- Developed new dimensions and angles for rocker-bogie utilizing a SOLIDWORKS equation feature and ensured its alignments

---

Name: Miguel Murillo

Email: [miguelmurill25@gmail.com](mailto:miguelmurill25@gmail.com)

- Designed and manufactured a mechanical slider system for suspension
- Integrated mounting points for the flyer dispenser, arm, and GPS
- Designed and built the chassis using 20x20 aluminum extrusions, HDPE sheets, and various 3D-printed components

---

Name: Luis Gama

Email: [luis.a.gama.diaz@gmail.com](mailto:luis.a.gama.diaz@gmail.com)

- Designed gear profiles for flyer dispenser
  - Modeled various rover parts, including wheels and LED pixel display mount
-



Name: Sarah Qu  
Email: [sarahqu411@gmail.com](mailto:sarahqu411@gmail.com)

- Designed electrical mounting board and miscellaneous chassis components
  - Arm assembly
- 

Name: Hannah Kim  
Email: [hannahkim2006@gmail.com](mailto:hannahkim2006@gmail.com)

- Designed and manufactured a bin to house flyers to be dispensed
  - Integrated bin assembly with chassis and arm
- 

Name: Ethan Choi  
Email: [ethanjchoiwork@gmail.com](mailto:ethanjchoiwork@gmail.com)

- Designed and manufactured rocker-bogie suspension joints for the rover
  - Ensured all parts of the suspension were properly toleranced
- 

### *Chapter 3.1: Flyer Dispenser*

The flyer dispenser is a 2DOF manipulator that can grab and dispense flyers via vacuum-powered suction cups. The dispenser sits at a stowed height of 196mm and has a dispensing height of 651mm.

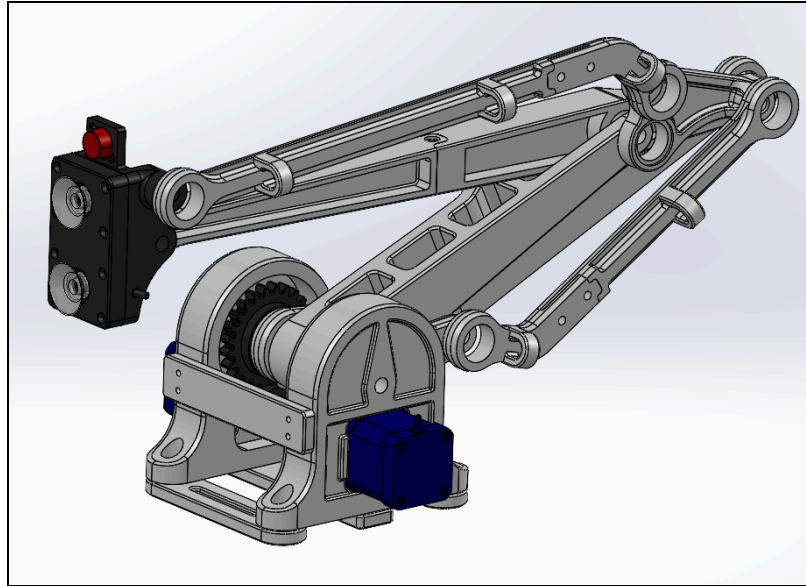


Figure 6: Assembly of Flyer Dispenser

The development of the flyer dispenser was inspired by a pick-and-place robot created by “ftobler” (<https://www.thingiverse.com/thing:1718984>). The linkages of the flyer dispensers were all resized to maximize dispensing height and minimize stowed height, while also constraining the stowed pose to sit within the chassis's frame. This optimization process was completed using a MATLAB script that iterates through thousands of linkage configurations until the aforementioned sizing was achieved. Simulations were also conducted through MATLAB's Simulink to find the torque requirements (Refer Fig 7:  $\sim 0.8\text{Nm}$  maximum when operating within dispensing bounds, ignoring value at time =  $\sim 2.7$  sec) of the flyer dispenser's lower and upper arms.

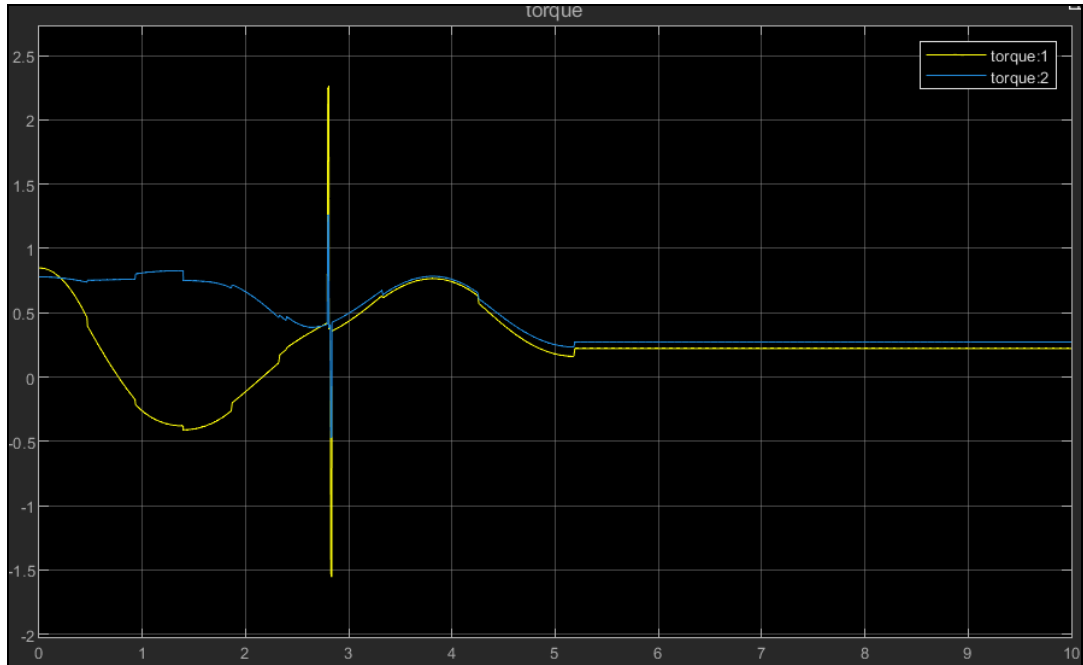


Figure 7: Simulated torque values (N-m) over time (sec) for Lower Arm (Yellow) and Upper Arm (Blue)

The manipulator consists of two main arms—the lower arm and upper arm—both of which revolve on the y-plane. The lower arm is directly connected to a drive gear for actuation, while the upper arm is connected to its drive gear via the parallel linkage set-up shown in Figure 8. This parallel linkage setup eliminates the need for mounting a heavy motor along the length of the arm. Both arms of the manipulator are actuated with 59 N-cm Nema 17 stepper motors (blue in Figure 6), which use ~3:1 gear ratios to provide the required torque needed for the dispensing motion. The Nema 17s also feature custom back-plates with integrated magnetic encoders, allowing us to accurately track the stepper motor positioning.

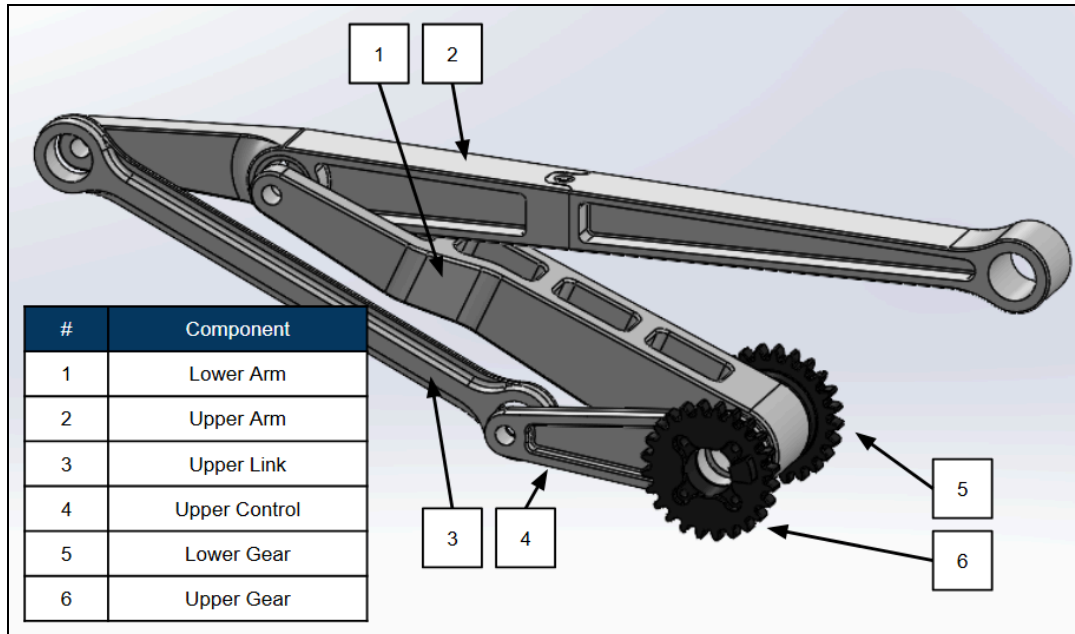


Figure 8: Lower Arm and Upper Arm linkage configuration

Keeping safety in mind, the manipulator's gears are housed by gearboxes built within the base of the manipulator. Limit switches are also mounted behind the gears, and are actuated by end-stops on the gears themselves. This allows the manipulator to be calibrated any time the limit switches are pressed (either during start-up or normal operation). The base contains multiple other end stops to stop the manipulator from over-extending itself in case of a fatal control error.

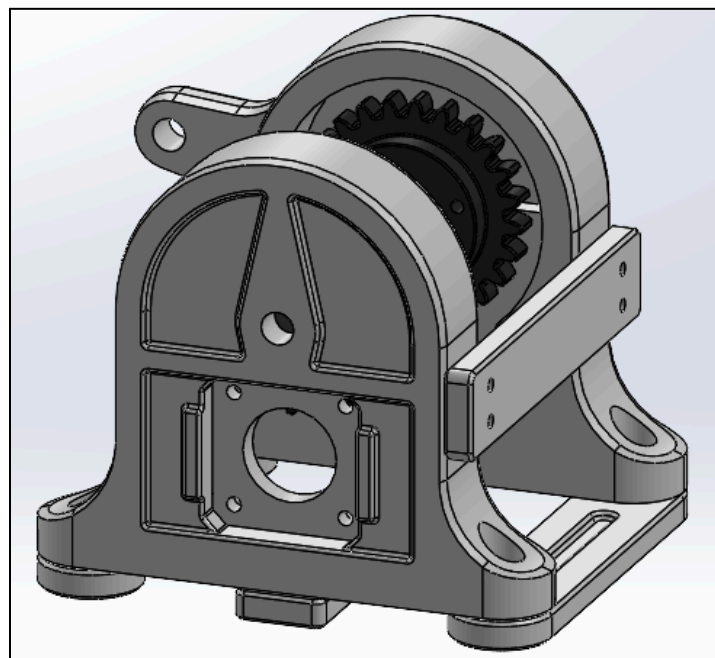


Figure 9: Sub-Assembly of Flyer Dispenser Base

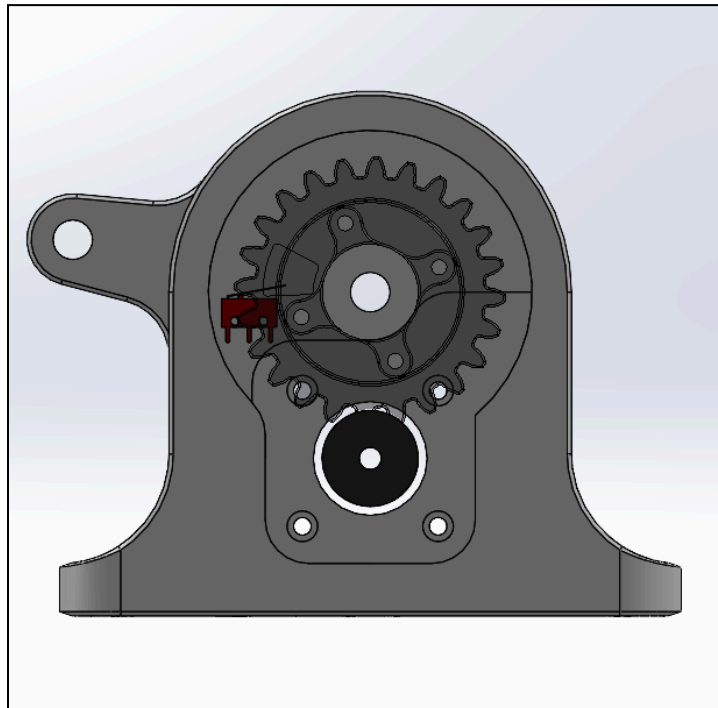


Figure 10: View of Limit Switch (red) and Lower Gear (transparent) interaction

The end-effector of the flyer dispenser, shown in Figure 11, is essentially an air manifold made of two 3D-printed parts that are joined together using heat-set inserts and airtight sealing tape. The orientation of the manipulator's end-effector remains fixed relative to the rover for seamless flyer retrieving and dispensing; which is accomplished through stabilizer arms linked in parallel to the actuated arms. An ultrasonic sensor (red in Figure 11) is mounted on the end-effector to sense if a flyer is attached.

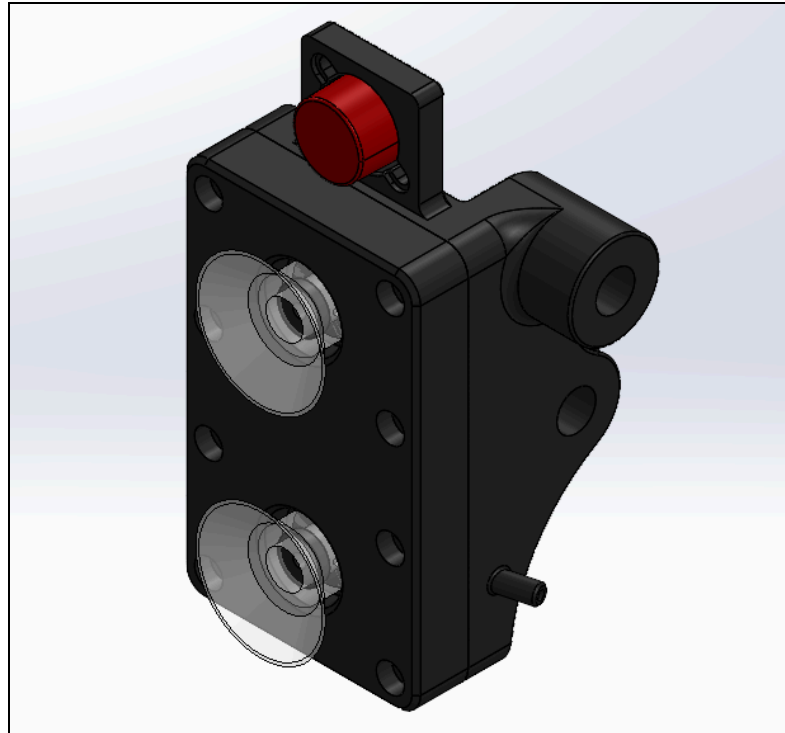


Figure 11: Sub-Assembly of End-Effector

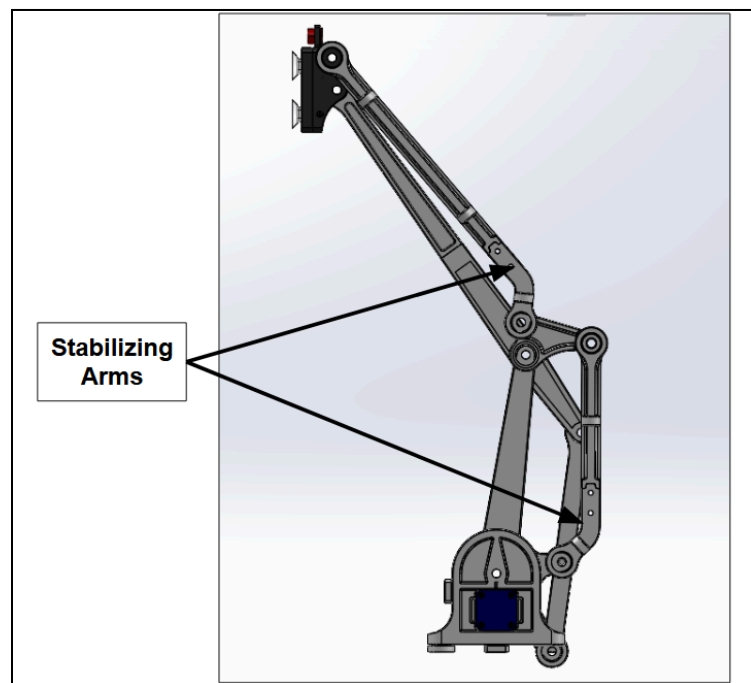


Figure 12. Diagram of Stabilizing Arms

Aside from hardware components, the entire manipulator is 3D printed with PETG filament, chosen due to its UV-resistant properties and consistent finishing.

8x22x7mm bearings are used at all of the joints for smooth movement, and M8 bolts of varying lengths are used as shafts for the bearings.

Manufacturing the current version of the flyer dispenser was largely successful, although a minor issue regarding the center of gravity (COG) became evident. Currently, the COG sits fairly behind the base in the dispenser's stowed position, causing the dispenser to torque back when no support is applied. This may not be an issue if the standoffs used to mount the base can hold against this torque since the COG sits over the middle of the rover. However, if needed, the weight of the arms and support links can easily be scaled back by reducing infill density when printing and by removing material from the design itself. The bearings can also be changed to fit M6 bolts instead, reducing the overall weight of our hardware—a great improvement since most of the hardware sits in the rear of the manipulator while stowed.



Figure 13. Current Flyer Dispenser prototype

### *Chapter 3.2: Chassis*

The chassis for this project was designed with dimensions of 300x500x200 mm, featuring a main frame composed of 20x20 aluminum extrusions to provide a robust and durable structure. The side exterior is constructed using HDPE plastic sheets to protect all electronic components. Frames on each face support the aluminum extrusions, while the front and rear sections feature a handle integrated extruded cut for the ease of lifting the robot without damaging other components.

The chassis also includes integrated mounting points for a flyer dispenser system. The top section of the chassis incorporates a mechanical slider utilizing a linear ball bearing attached to a 20x20 extrusion for added support and stability. Additionally, the arm assembly is mounted onto two parallel 20x20 extrusions to ensure structural integrity. The rear of the chassis features a GPS mount, strategically placed in order to maximize the distance from electronics that could interfere with the signal.

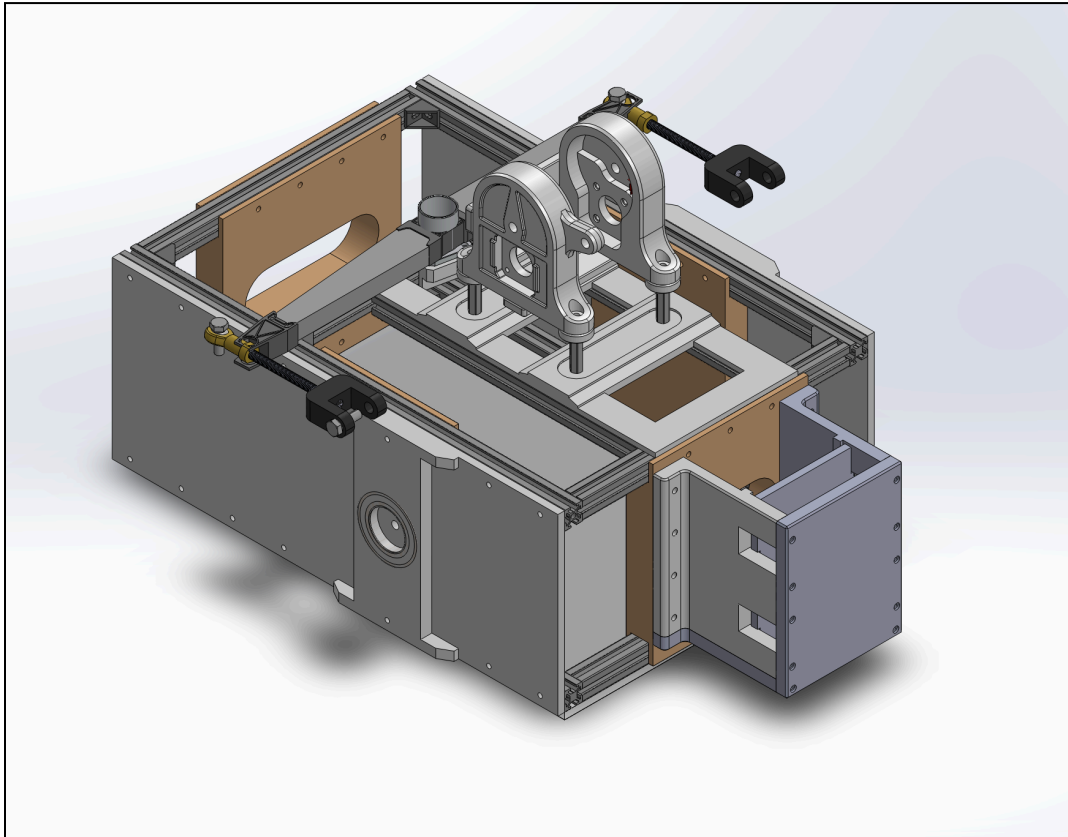


Figure 14: Assembly of Chassis Components



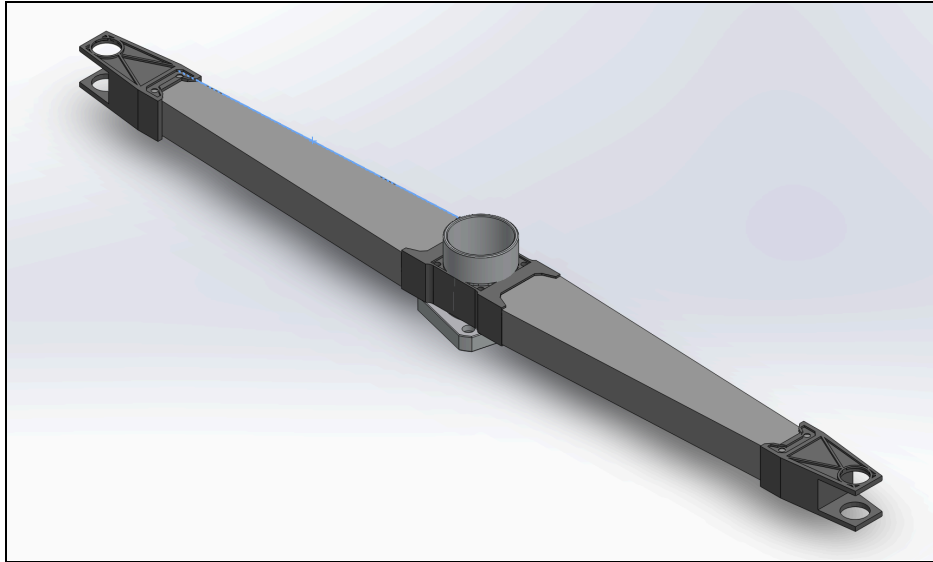


Figure 15: Assembly of Mechanical Slider for suspension, utilizing a linear ball bearing

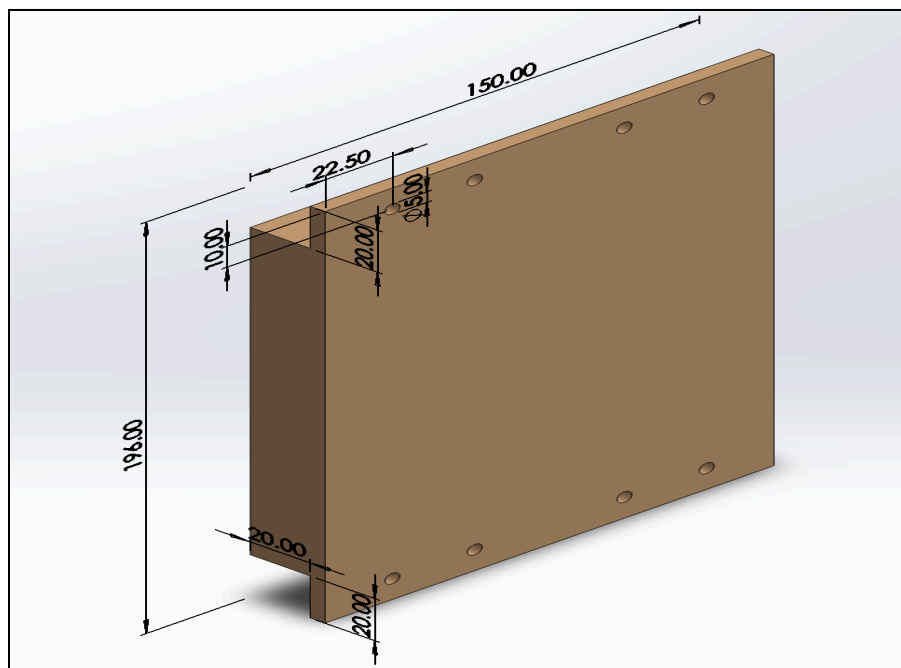


Figure 16: Side Support for 20x20 extrusions

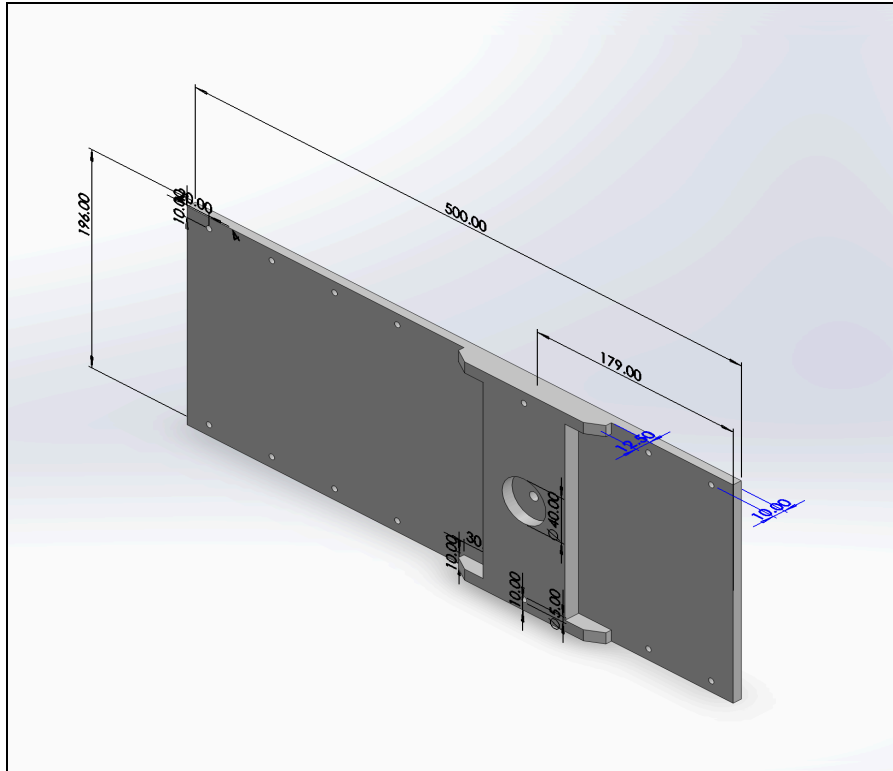


Figure 17: 500mm Side Panel

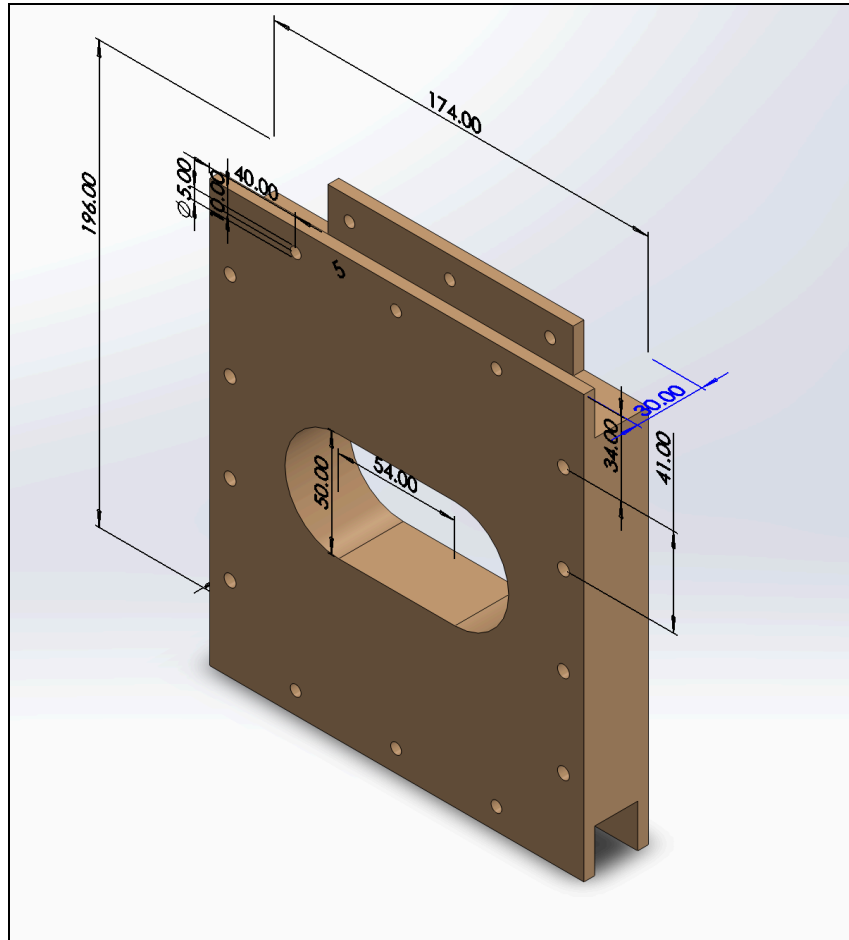


Figure 18: Front Support with bin mounting

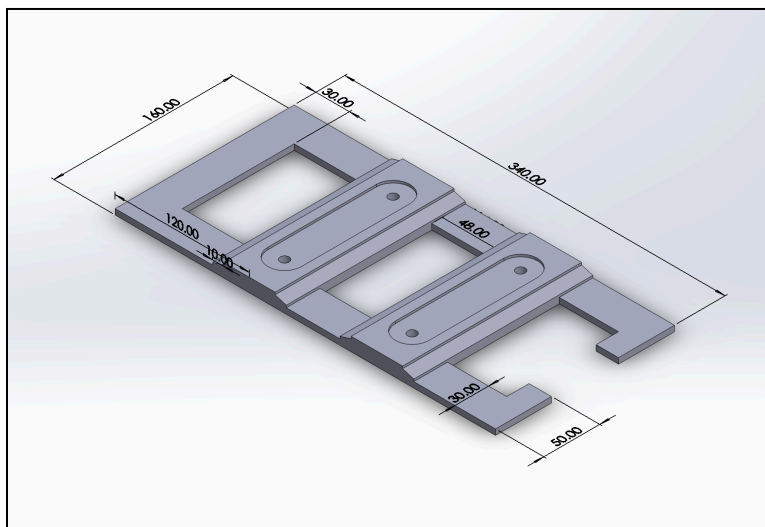


Figure 19: Arm mounting bracket

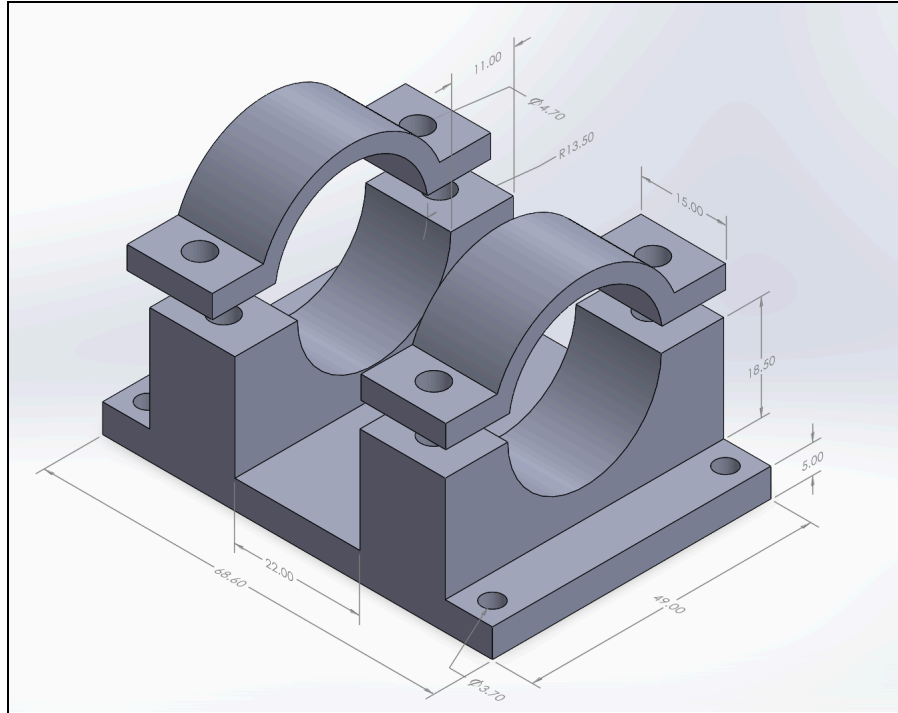


Figure 20: Pump Mounting Plate and Collar

## Chapter 3.3: Suspensions

### Sketch & Dimensions

The rocker-bogie-inspired suspension system in this project was designed with unique dimensions, developed using the equation feature in SOLIDWORKS, and based on the chassis dimensions of 300x500x200 mm. The rocker-bogie mechanism, commonly used in planetary rovers, provides six degrees of freedom (DOF) for enhanced terrain adaptability, allowing the system to traverse uneven surfaces while maintaining stability. It utilizes an in-wheel motor configuration for independent wheel control and integrates servo motors to regulate movement and articulation. The design process began with an initial sketch, where dimensions were imported and controlled using equations to maintain parametric integrity. The front and top views were assigned distinct values, particularly for angles, ensuring adaptability and accuracy. These views were then interconnected using the 3D sketch feature, allowing for a cohesive and well-defined assembly that maintains the desired motion and stability of the suspension system.

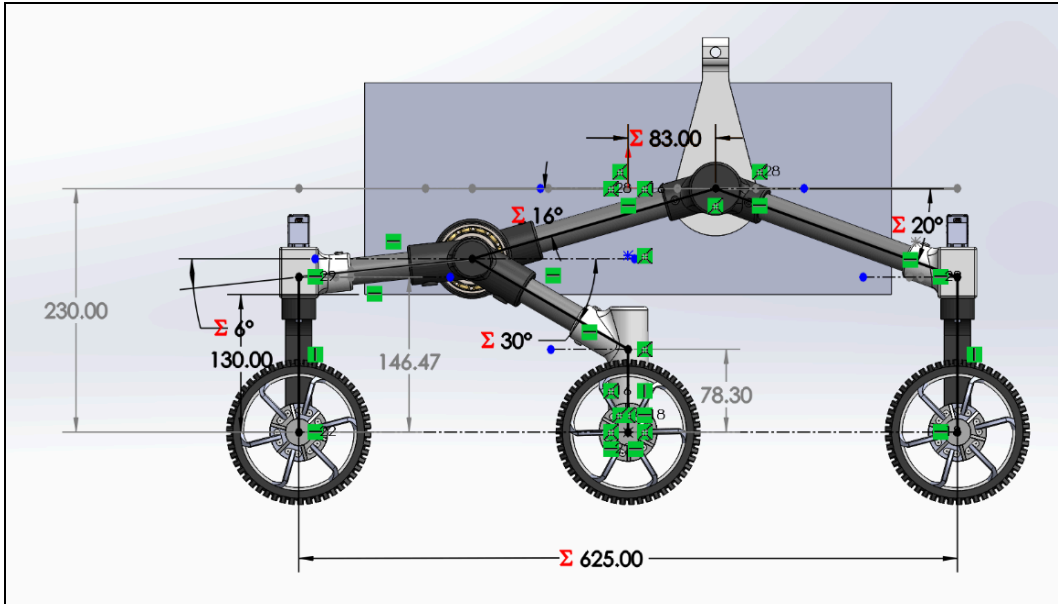


Figure 21: Front View Sketch of Rocker-Bogie Suspension

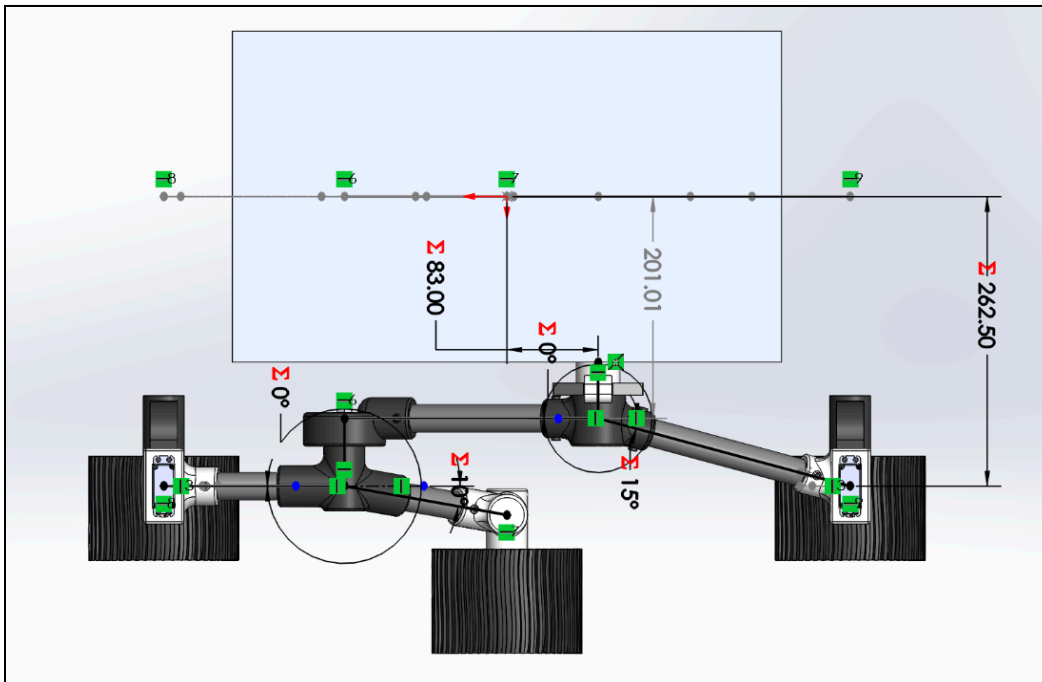


Figure 22: Top View Sketch of Rocker-Bogie Suspension

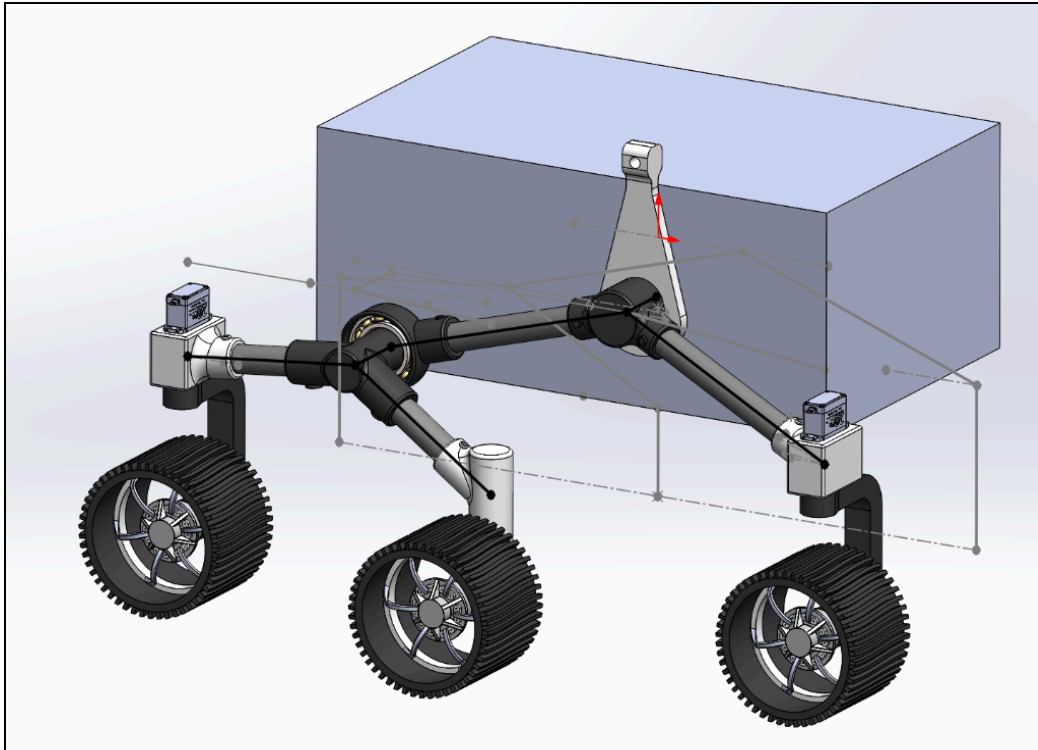


Figure 23: 3D Sketch Referenced by Top and Front Sketches

## Mounts & Joints 3D Design

The dimensions of each suspension component were derived from the initial sketch as a reference.

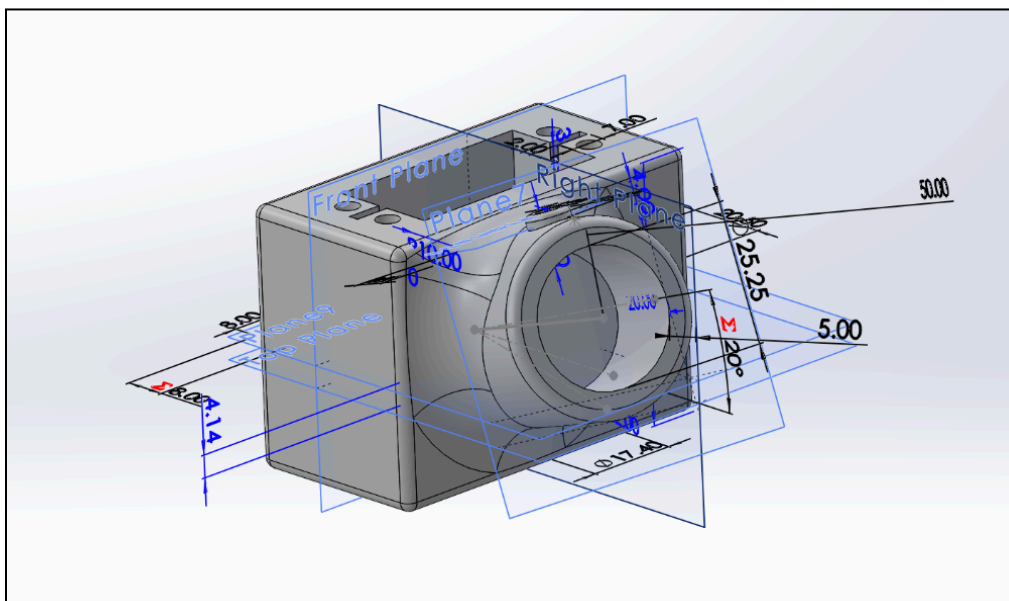


Figure 24: Servo mount

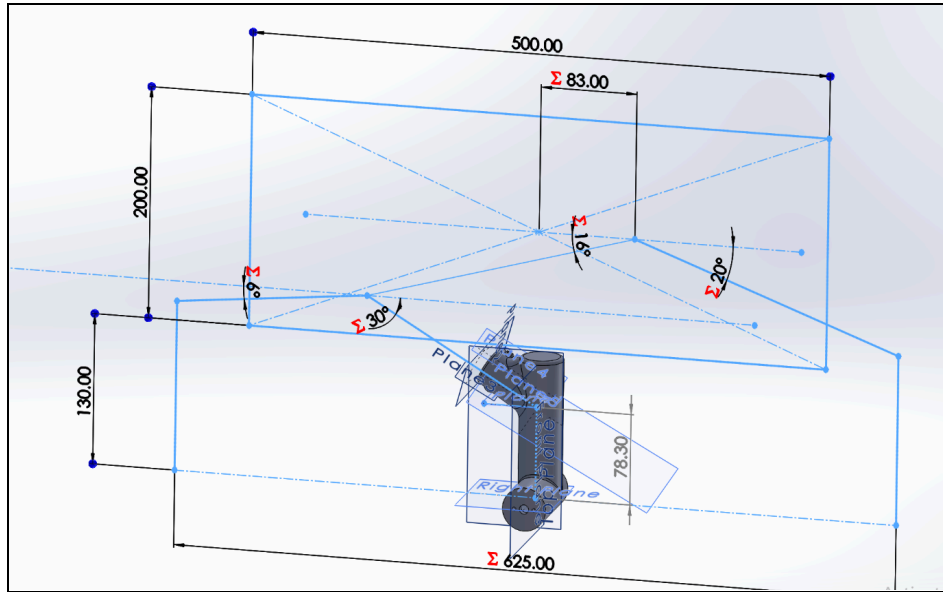


Figure 25: Middle In-Wheel Motor Mount

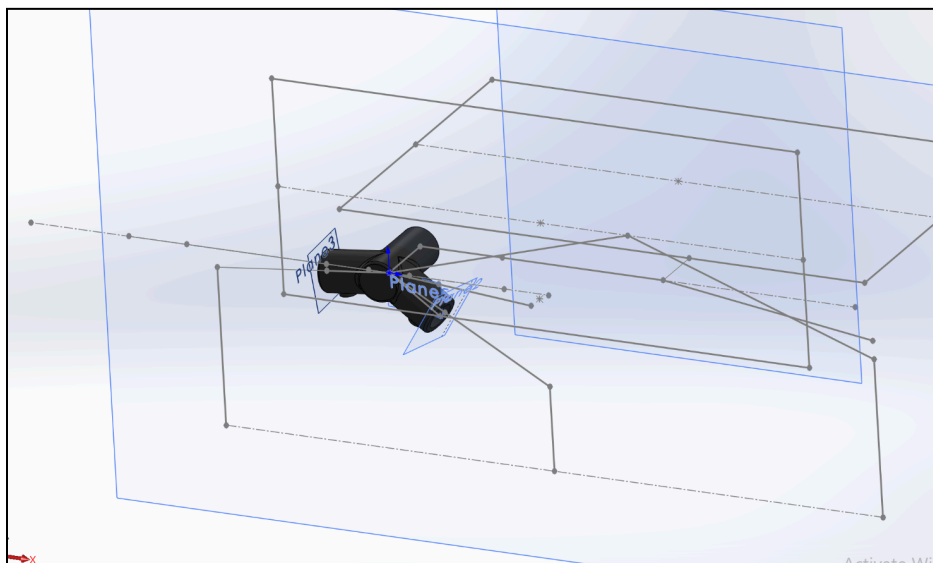


Figure 26: Rear Joint Connecting Rear and Middle Wheels

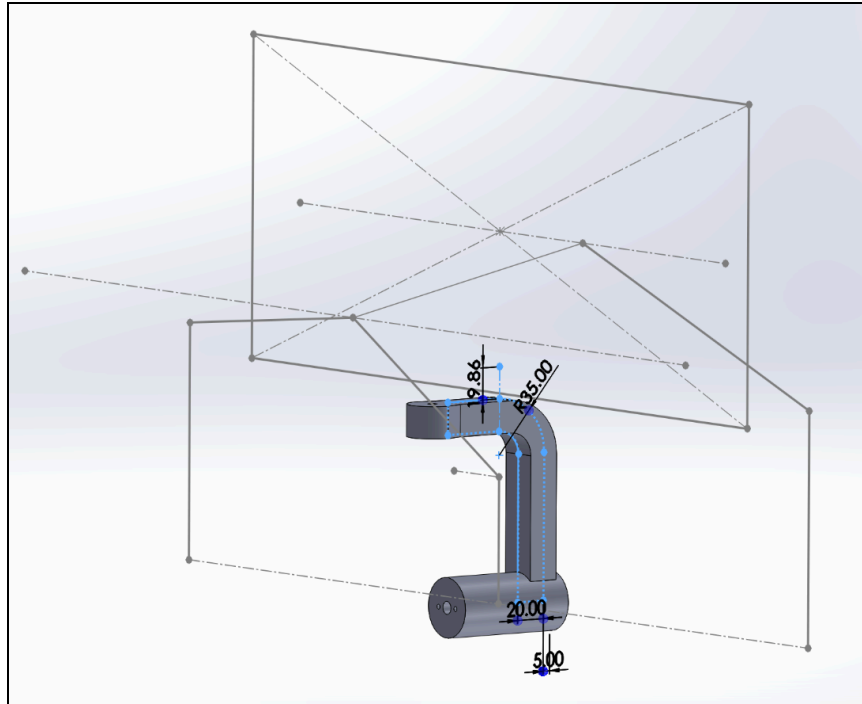


Figure 27: Front and Rear In-Wheel Motor Mount

### Rocker-Bogie Mechanisms

The in-wheel motor will operate using a 6mm flange coupling.

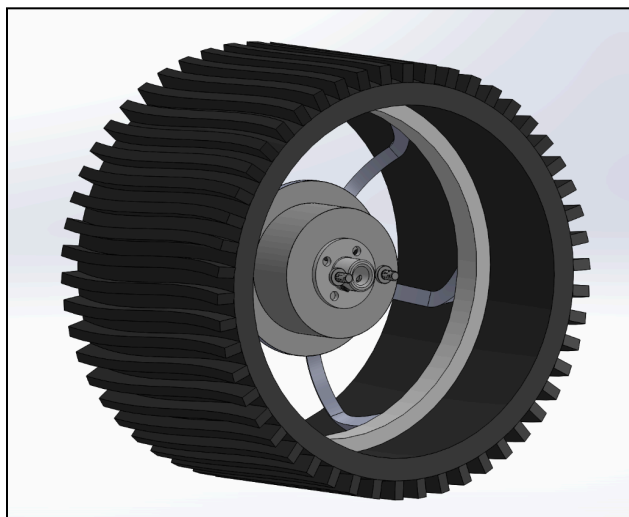


Figure 28: Rover Wheel and a Flange Coupling

The 360-degree in-place rotation will operate using the servo.



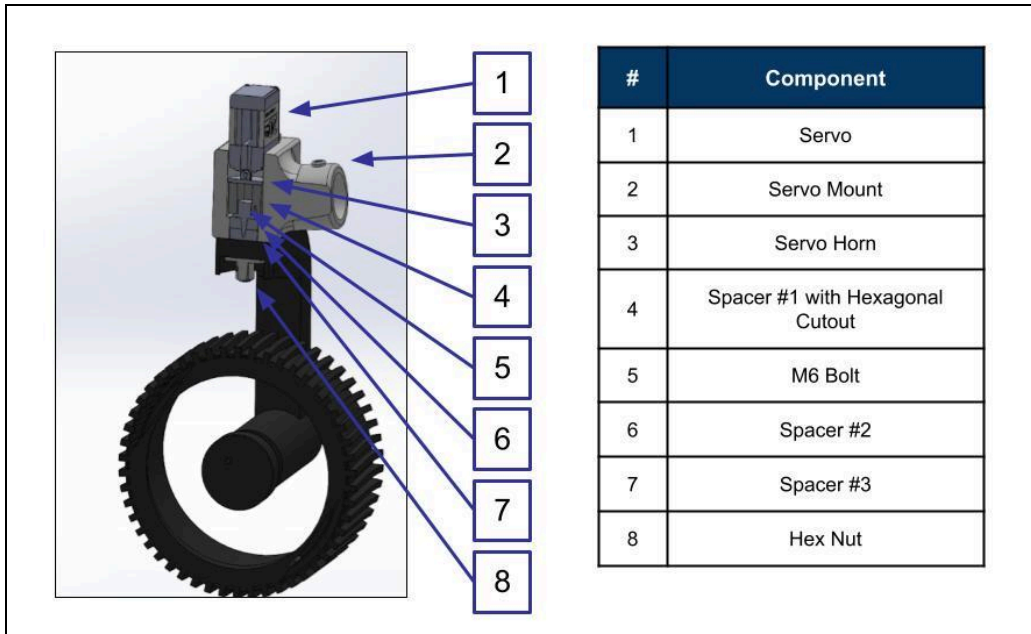


Figure 29: Section View of the Servo Mount

The joint will be press-fitted with adhesive to the rocker.

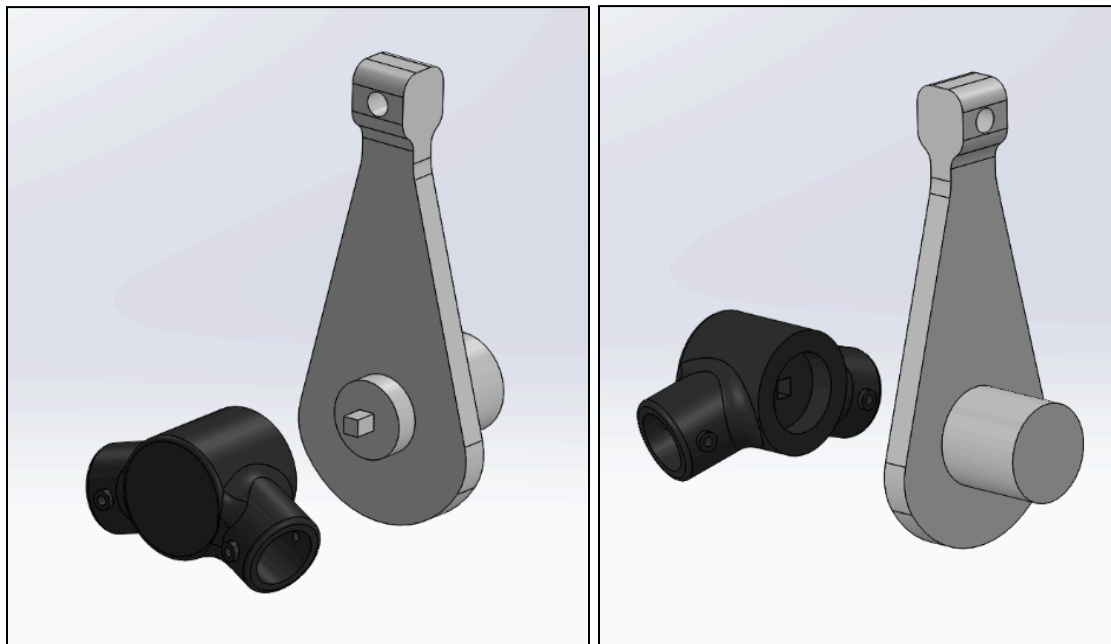


Figure 30: Exploded View of the Rocker Assembly (Front and Back)

The 40 mm inner diameter and 68 mm outer diameter bearings connect the rocker to the chassis and connect the rear joints allowing movement. 25 mm outer diameter carbon fiber rods will be held in place using M3 set screws.

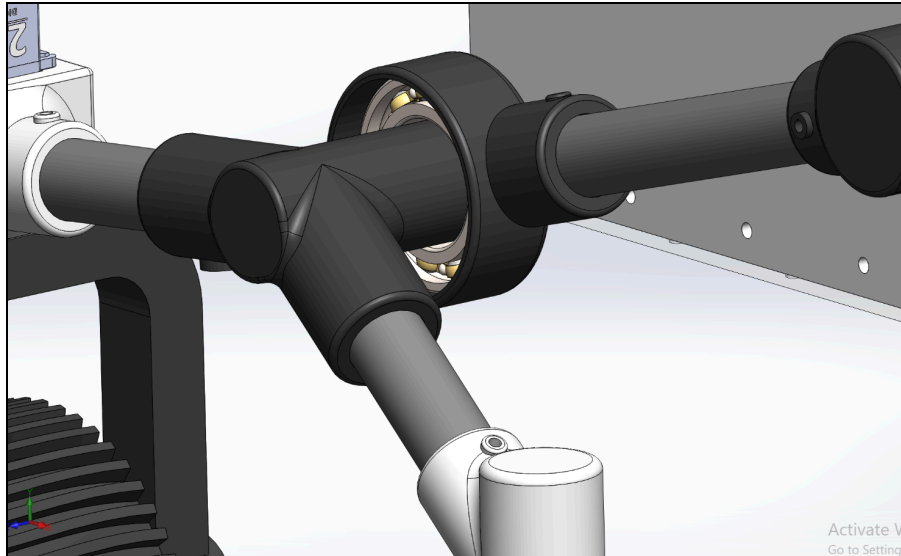


Figure 31: Bearing Connecting Two Joints

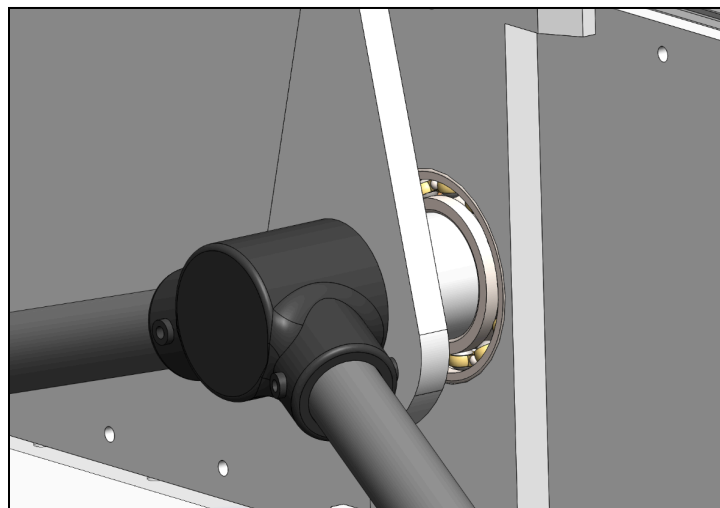


Figure 32: Bearing Connecting Suspensions and Chassis

The Differential and rocker will be connected using an M8-120 threaded rod and an 8mm inner diameter heim joint mounted using M8 bolts.

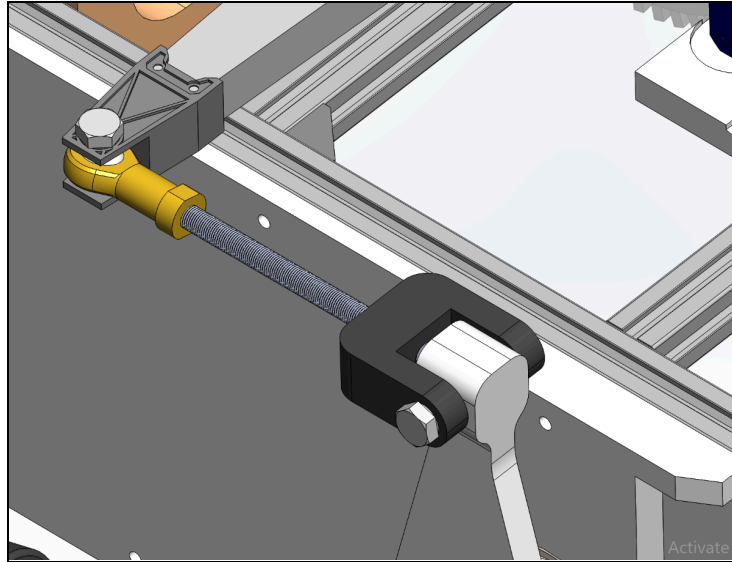


Figure 33: Differential and Rocker Connection

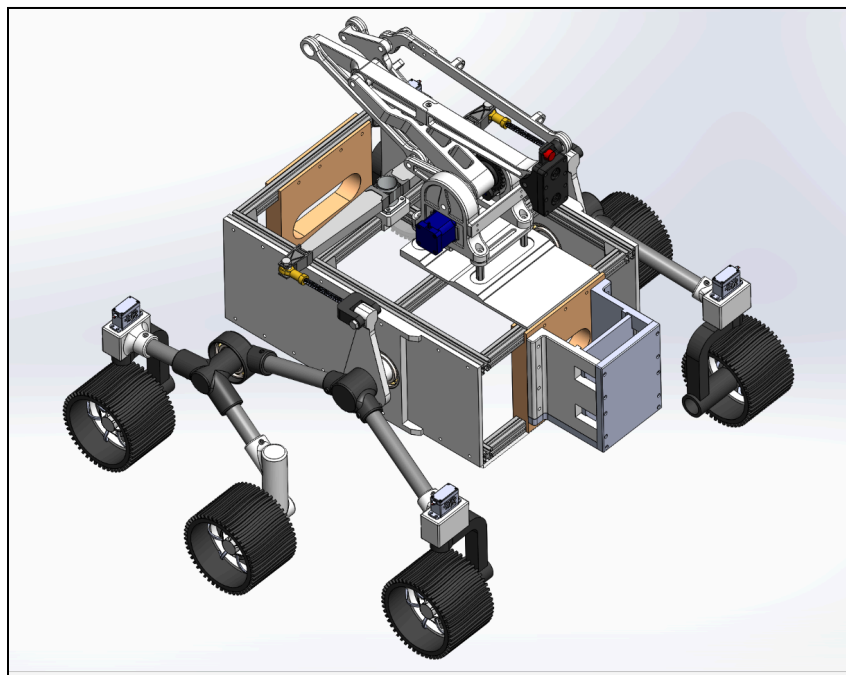


Figure 34: Suspension Integrated with Chassis

## Chapter 4: Controls Subteam

Name: Nikolaj Kim

Email: [nikolajkim@gmail.com](mailto:nikolajkim@gmail.com)

- Developed NLP using conversational AI software Rasa to create an AI agent that can effectively communicate with potential rushees regarding Theta Tau.
- Integrated Text-To-Speech (TTS) with the NLP using Piper, so the robot could smoothly converse with the user.

---

Name: Wilson Nguyen

Email: [wilsonnguyen0928@gmail.com](mailto:wilsonnguyen0928@gmail.com)

- Developed arm and LiDAR sensor in for the URDF model using Xacro for simulation testing and visualizing path planning
- Implemented SLAM Toolbox for autonomous navigation

---

Name: Jake Silverman

Email: [jsilverman54@gmail.com](mailto:jsilverman54@gmail.com)

- Developed wake-word functionality using Porcupine for the AI agent to be listening in the background and to engage its conversation functionality when called upon with the wake-word
  - Integrated speech-to-text models such as OpenAI's Whisper and Mozilla's DeepSearch into the robot's NLP and Text-to-Speech functionality to listen and respond to rushees
-

Name: Tyler Buffa  
Email: [tylerbuffa@gmail.com](mailto:tylerbuffa@gmail.com)

- Developed custom-made ros2\_control controller for driving/steering rover
- Wrote ros2\_control hardware interfaces for motors and magnetic encoders
- Developed rover, wheel, and arm URDF files
- Implemented manipulator motion planning using the MoveIt2 framework and Open Motion Planning Library (OMPL), along with an inverse kinematics solver for cartesian-space motion.

---

## Chapter 4.1: Rover Control System

The control loop for our rover was built using the ROS2 framework. This framework provides a wide variety of prebuilt packages that contain solutions for common robotics problems, eliminating the need to develop code for each part of the rover. Furthermore, each package boils down to using the same systems for intrasystem communications, allowing seamless integration between the various packages. Another advantage of using ROS2 is its built-in simulation capabilities, allowing robot development even before the physical build is complete. Using a Unified Robot Description File (URDF), we are able to generate a simulated model of the robot and test it in simulated worlds. For this project, Gazebo was chosen as the simulation software due to its ease of use and helpful community. The URDF also allows us to track how the RushBot changes over time via transformation frames, which record the position and orientation of every joint/link of the robot.

### Low-Level

The core of controlling RushBot lies in a package called ros2\_control, a framework that tackles two of the biggest problems when designing a robot: interfacing hardware and controlling hardware. Firstly, ros2\_control's hardware interfacing is used to provide communication between ROS and RushBot's many actuators and sensors. Secondly, ros2\_control provides a platform for designing and managing controllers, which allows RushBot to command its actuators and read sensor states via the hardware interfaces. RushBot uses two different controllers for movement: a custom-made rover controller and a joint trajectory controller.

The rover controller's role is self-explanatory—it commands the motors and servos to steer/drive the rover. More specifically, the controller receives velocity commands from an outside source and outputs actuator interface commands to achieve the desired motion. Another feature of the controller is position tracking. For this aspect, the rover controller uses the wheel states read by the magnetic encoders to generate an odometry frame, which records the position and orientation of the rover with respect to the fixed world frame (a frame to which all rover movement is referred). One of the issues of designing a car-like controller is mitigating wheel slip, which occurs when wheels are forced to move laterally. To deal with this issue, the rover controller implements Ackermann steering geometry to ensure each wheel drives around the same center of turning. In the future, the rover controller will also generate pivoting commands, so that the rover may turn toward a desired direction without the need to move forward.

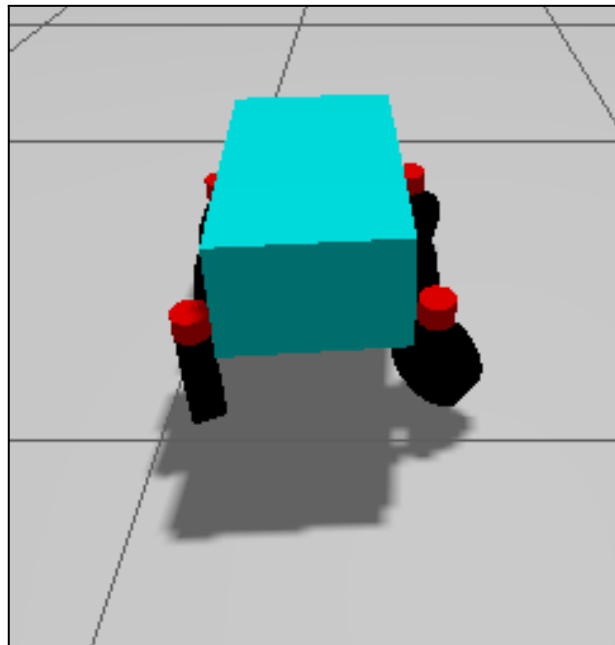


Figure 35: Gazebo Rover Controller simulation—inner tires (right) have steeper turning angle relative to outer tires (left) when using Ackermann steering

The joint trajectory controller is used to command the motors of the flyer dispenser. Similar to the rover controller, it receives velocity commands and outputs actuator interface commands. This controller works especially well with manipulators like our flyer dispenser since it can generate smooth, minimal jerk motion. This joint trajectory controller will eventually be chained with a PID controller, which will ensure accurate positioning regardless of human-generated error (e.g. bumping into the arm).

## Mid-Level

Working one layer above `ros2_control` are the packages that process sensor data, which allow our rover to work autonomously. One of the main features of RushBot is a LiDAR sensor that can range-find points across a 2D plane. To use the LiDAR to the fullest extent, we implemented grid-map based Simultaneous Localization and Mapping (SLAM) through the package SLAM Toolbox. By integrating sensor fusion and gathering sensor data from a LiDAR and IMU, the robot is able to map out its surroundings and localize itself relative to its environment.

SLAM toolbox has several different modes and the one we chose to run SLAM on is Online Asynchronous. Online means that the data is being processed in real-time rather than some previous data and Asynchronous means that it always processes the most recent scan to avoid lagging, even if it has to skip some scans. SLAM itself relies on sensor data, in this case, LiDAR and IMU data, to estimate the robot's location and map its surroundings. The process involves two main components: Localization which is how the robot determines its position using landmarks and Mapping which is when the robot constructs or updates a map based on incoming sensor data. The two go hand-in-hand for real-time autonomous navigation.

In order for the robot to visualize its surroundings, we need to create a LiDAR sensor in an URDF file to be deployed in the Gazebo simulation. From there, the robot is able to “see” its surroundings, which are marked by red dots in RViz (Fig. 36 and Fig. 37).

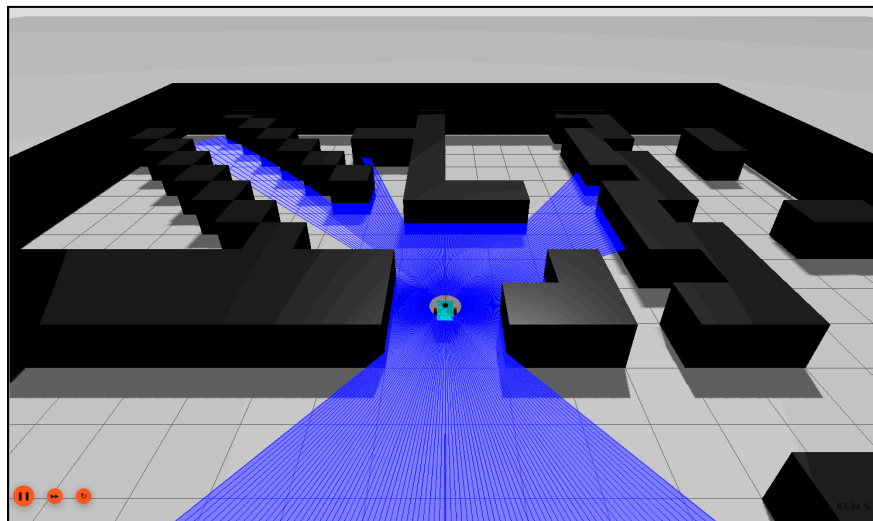


Figure 36: Gazebo LiDAR simulation—LiDAR rays are represented by the blue lines emitted from the LiDAR link

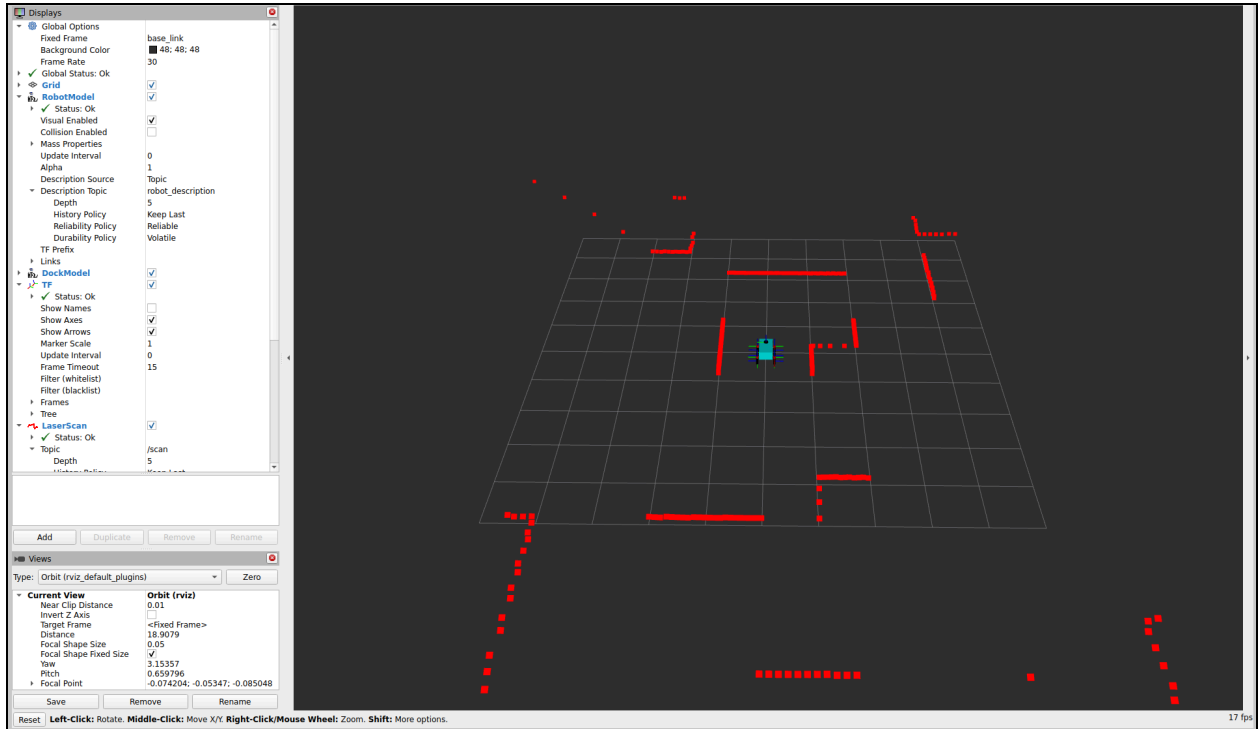


Figure 37: RViz LiDAR Visualization—RViz marks the point measured by each LiDAR ray with a red dot

```

}
linear_acceleration {
  x: -8.4891675928214e-08
  y: 2.3271281840135014e-11
  z: 9.8000000000918455
}
linear_acceleration_covariance {
  data: 0
  data: 0
  data: 0
  data: 0
  data: 0
  data: 0
  data: 0
  data: 0
  data: 0
}

header {
  stamp {
    sec: 120
  }
  data {
    key: "frame_id"
    value: "RushBotT::base_footprint::imu_sensor"
  }
  data {
    key: "seq"
    value: "120"
  }
}
entity_name: "RushBotT::base_footprint::imu_sensor"
orientation {
  x: 3.3914793888103608e-16
  y: 4.2859023742810512e-09
  z: 3.5207207536103397e-16
  w: 1
}
orientation_covariance {
  data: 0

```

Figure 38: IMU message packet



To further improve overall localization accuracy, IMU data is fused with LiDAR localization. The IMU provides orientation and acceleration data which is integrated over time to estimate total displacement. Fig. 38 illustrates what the IMU message packet looks like.

Our next goal is to fuse the IMU data using an Extended Kalman Filter with the LiDAR to get more accurate mapping and localization for the robot. Following that goal will be the implementation of GPS sensor data, further improving localization and giving RushBot the ability to patrol regions via waypoints.

## High-Level

At the highest level, RushBot employs two motion planning packages, MoveIt2 and Nav2, to guide the rover and flyer dispenser to their target positions.

MoveIt2 provides RushBot with the paths the manipulator must take to accomplish flyer dispensing. It creates these paths by first reading the current manipulator joint-space states, then generating way-points to a desired end-effector pose using the Open Motion Planning Library (OMPL). To have the end-effector move along points in the cartesian-space rather than the joint-space, an inverse kinematics solver is used during way-point generation as well. Moreover, MoveIt2 will plan this path with the geometry of the rover in mind — fed in by the URDF — to ensure no collisions or over-extensions will occur. Once these paths are generated, MoveIt2 gives commands to the joint trajectory controller so the manipulator may move.

Analogous to MoveIt2, Nav2 provides RushBot with paths the rover should take to achieve a certain pose from its current pose; this is given to RushBot from the localization that happens in the mid-level packages. Nav2 has not yet been implemented into RushBot, but we are planning to use the GPS waypoints as the desired pose inputs so that the rover can patrol a designated area. We also intend to incorporate LiDAR sensor data to enhance safety during both Nav2 and MoveIt 2 motion planning.

## Chapter 4.2: Conversational Interface

We also focused on developing the rover's capability to interact with users via speech. The primary objective was to develop a conversational interface that allows the robot to recognize and respond to spoken queries, specifically related to the Theta Tau all without the necessity of an internet connection.

To accomplish this, we implemented a multi-step approach covering Speech-To-Text (STT), Natural Language Understanding (NLU), and Text-To-Speech (TTS).

The first step in enabling speech interaction was converting the spoken input from users into text. For this, we utilized OpenAI's open-source Whisper. Whisper is an automatic speech recognition system (ARS) that is trained on hundreds of thousands of hours of supervised data collected from the internet. By using this ARS to transcribe queries in real-time, the robot is able to immediately process spoken queries and pass transcriptions to the next step of processing.

Once the spoken query is transcribed into text, the Rasa framework processes and understands the user's input. Rasa is an open-source conversational AI platform that allows us to build contextual dialogue systems. We created a custom Rasa model specifically tailored to answering questions about the Theta Tau and our chapter. This model was trained to recognize various intents and entities related to the organization, allowing the bot to respond appropriately.

Once Rasa generates a response, the Piper TTS system converts the text response back into speech. The robot is then able to speak to the user, creating a conversational loop.

The ability to converse with the robot adds an extra level of accessibility, allowing users to inquire about anything regarding Theta Tau.

## Chapter 5: Impact Assessment and Conclusion

Although RushBot is not yet fully completed, our team has made significant progress toward completing this project. Over the past quarter, we have successfully developed and integrated key subsystems, including the mechanical arm for flyer distribution, the AI chatbot for interactive engagement, and the foundational electrical framework supporting high and low-voltage operations. With these major components in place, we are on track to complete the full autonomous rover within the coming quarter.

Beyond technical progress, this project has been a valuable learning experience for our team members. RushBot has provided hands-on opportunities to develop skills in robotics, mechanical design, embedded systems, software development, and AI integration.

Looking ahead, we anticipate RushBot will have a significant impact on our fraternity's recruitment efforts. By introducing an interactive and engaging element to boothing events, we expect to attract more traffic and increase participation in our rush and boothing events.

Ultimately, while not completed, RushBot has already proven to be an excellent learning platform for our members, and as we refine and enhance it, we expect it to leave a lasting impact on our fraternity.